

Kernel Methods

Sanjay Singh

School of Computer Engineering
Manipal Institute of Technology, MAHE, Manipal-576104, INDIA
sanjay.singh@manipal.edu

February 23, 2026

Sanjay Singh

Kernel Methods

Feature Maps

- Recall the problem of predicting the price of a house (denoted by y) from the living area (denoted by x) with linear regression
- There we were suppose to fit a linear function of x to the training data
- What if the price y can be more accurately represented as a *non-linear* function of x ?
- To do so, we need a more expressive family of models than linear models

Sanjay Singh

Kernel Methods

- Let us consider fitting cubic functions $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$
- We can view the cubic function as a linear function over a different set of feature variables, defined below
- Let the function $\phi : \mathbb{R} \rightarrow \mathbb{R}^4$ be defined as

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \in \mathbb{R}^4.$$

- Let $\theta \in \mathbb{R}^4$ be the vector containing $\theta_0, \theta_1, \theta_2, \theta_3$ as entries
- We can rewrite the cubic function in x as

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 = \theta^T \phi(x)$$

- Thus, a cubic function of x can be viewed as a linear function over the variable $\phi(x)$

Attributes vs. features

- We'll call the “original” input value the **attributes** of a problem (in this case x , the living area)
- When the original input is mapped to some new set of quantities $\phi(x)$, we call those new quantities the **feature** variables
- We call ϕ a **feature map**, which maps the attributes to features

- We'll derive the gradient descent algorithm for fitting the model $\theta^T \phi(x)$
- Recall that in ordinary least square problem we fit $\theta^T x$, and the batch gradient descent update is given by

$$\begin{aligned}\theta &:= \theta + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)}))x^{(i)} \\ &:= \theta + \alpha \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})x^{(i)}\end{aligned}$$

- Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ be a feature map that maps attribute $x \in \mathbb{R}^d$ to the features $\phi(x) \in \mathbb{R}^p$
- Our goal is to fit function $\theta^T \phi(x)$, with θ being a vector in $\mathbb{R}^p$ instead of $\mathbb{R}^d$
- We can now replace all the occurrence of $x^{(i)}$ in above algorithm by $\phi(x^{(i)})$ to obtain the new update rule

$$\theta := \theta + \alpha \sum_{i=1}^m (y^{(i)} - \theta^T \phi(x^{(i)}))\phi(x^{(i)})$$

- The corresponding stochastic gradient descent update is given by

$$\theta := \theta + \alpha (y^{(i)} - \theta^T \phi(x^{(i)}))\phi(x^{(i)})$$

Kernel Function

- **Briefly speaking:** a kernel is a shortcut that help us do certain calculation faster which otherwise would involve computations in higher dimensional space
- **Mathematical definition:** $K(x, y) = \langle f(x), f(y) \rangle$, K is the kernel function. $x, y \in \mathbb{R}^n, f : \mathbb{R}^n \mapsto \mathbb{R}^m, m \gg n$

Intuition¹

Normally calculating $\langle f(x), f(y) \rangle$ require us to calculate $f(x), f(y)$ first, and then do the dot product. These two computation steps can be quite expensive as they involve manipulations in m dimensional space, where m can be a large number. But after all the trouble of going to the high dimensional space, the result of the dot product is really a scalar: we come back to one-dimensional space again! Now, the question is: do we really need to go through all the trouble to get this one number? Do we really have to go to the m -dimensional space? The answer is no, if you find a clever kernel.

¹Lili Jiang's blog

- $x = [x_1 \ x_2 \ x_3]^T$, $y = [y_1 \ y_2 \ y_3]^T$, then for the function

$$f(x) = [x_1x_1 \ x_1x_2 \ x_1x_3 \ x_2x_1 \ x_2x_2 \ x_2x_3 \ x_3x_1 \ x_3x_2 \ x_3x_3]^T$$

, the kernel is $K(x, y) = (\langle x, y \rangle)^2$

- Let $x = [1 \ 2 \ 3]^T$, $y = [4 \ 5 \ 6]^T$

$$f(x) = [1, 2, 3, 2, 4, 6, 3, 6, 9]^T$$

$$f(y) = [16, 20, 24, 20, 25, 30, 24, 30, 36]^T$$

$$\langle f(x), f(y) \rangle =$$

$$16 + 40 + 72 + 40 + 100 + 180 + 72 + 180 + 324 = 1024$$

- A lot of computation because $f : \mathbb{R}^3 \mapsto \mathbb{R}^9$
- Now lets use kernel,
 $K(x, y) = (4 + 10 + 18)^2 = 32^2 = 1024!$ WoW
 Same result, but this calculation so much easier
- The word "kernel" is used in mathematics to denote a weighting function for a weighted sum or integral.

LMS with kernel trick

- The BGD and SGD becomes computationally expensive when the features $\phi(x)$ is high-dimensional
- Suppose $x \in \mathbb{R}^d$, and let $\phi(x)$ be the vectors containing all the monomials of x with degree ≤ 3

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_2x_1 \\ \vdots \\ x_1^3 \\ x_1^2x_2 \\ \vdots \end{bmatrix}$$

- Dimension of features $\phi(x)$ on the order of d^3
- It is prohibitively long vector for computational purpose, say for $d = 1000$, each update requires at least computing and storing $1000^3 = 10^9$ dimensional vector
- 10^9 is 10^6 time slower than the update rule for ordinary LMS

- We initialize $\theta = 0$, and focus on iterative update

$$\theta := \theta + \alpha \sum_{i=1}^m (y^{(i)} - \theta^T \phi(x^{(i)})) \phi(x^{(i)})$$

- At any time, θ can be represented as a linear combination of vectors $\phi(x^{(1)}), \dots, \phi(x^{(m)})$

- We can show this inductively, at initialization,

$$\theta = 0 = \sum_{i=1}^m 0 \cdot \phi(x^{(i)})$$

- Assume at some point, θ can be represented as $\theta = \sum_{i=1}^m \beta_i \phi(x^{(i)})$
for some $\beta_i \in \mathbb{R}$

- We claim that in the next round, θ is still a linear combination of $\phi(x^{(1)}), \dots, \phi(x^{(m)})$, because

$$\begin{aligned} \theta &:= \theta + \alpha \sum_{i=1}^m (y^{(i)} - \theta^T \phi(x^{(i)})) \phi(x^{(i)}) \\ &= \sum_{i=1}^m \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^m (y^{(i)} - \theta^T \phi(x^{(i)})) \phi(x^{(i)}) \\ &= \sum_{i=1}^m \underbrace{(\beta_i + \alpha(y^{(i)} - \theta^T \phi(x^{(i)})))}_{\text{new } \beta_i} \phi(x^{(i)}) \end{aligned}$$

- The strategy is to implicitly represent the p -dimensional vector θ by a set of coefficients $\beta_1, \dots, \beta_m$
- On doing this we derive the update rule for β s

$$\beta_i := \beta_i + \alpha(y^{(i)} - \theta^T \phi(x^{(i)}))$$

- $\beta_i := \beta_i + \alpha(y^{(i)} - \theta^T \phi(x^{(i)}))$
- Here, we still have the old θ on RHS
- Replacing θ by $\theta = \sum_{j=1}^m \beta_j \phi(x^{(j)})$ yields

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^m \beta_j \phi(x^{(j)})^T \phi(x^{(i)}) \right) \quad \forall i \in \{1, \dots, m\}$$

- We often write $\phi(x^{(j)})^T \phi(x^{(i)})$ as $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ to emphasize that it is the inner product of two feature vectors
- Viewing β_i 's as the new representation of θ , we have translated the BGD into an algorithm that updates β iteratively

- It appears that at every iteration, we still need to compute $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$, which may take $O(p)$ operations
- Following properties helps
 - We can pre-compute the pairwise inner product $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ for all pairs of i, j before loop starts
 - For the feature map $\phi(x)$, computing $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ can be efficient and does not require computing $\phi(x^{(i)})$ explicitly
- This is because

$$\begin{aligned} \langle \phi(x), \phi(z) \rangle &= 1 + \sum_{i=1}^d x_i z_i + \sum_{i,j \in \{1, \dots, d\}} x_i x_j z_i z_j + \sum_{i,j,k \in \{1, \dots, d\}} x_i x_j x_k z_i z_j z_k \\ &= 1 + \sum_{i=1}^d x_i z_i + \left(\sum_{i=1}^d x_i z_i \right)^2 + \left(\sum_{i=1}^d x_i z_i \right)^3 \\ &= 1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3 \end{aligned}$$

- So, to compute $\langle \phi(x), \phi(z) \rangle$, we first compute $\langle x, z \rangle$ with $O(d)$ time and take another constant number of operation to compute $1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3$

- We define the **Kernel** corresponding to the feature map ϕ as a function that maps $\mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ satisfying:

$$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$$

- We can write the final algorithm as follows:

1 Compute all the values $K(x^{(i)}, x^{(j)}) \triangleq \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$ for all $i, j \in \{1, \dots, m\}$. Set $\beta := 0$

2 **Loop:**

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^m \beta_j \phi(x^{(j)})^T \phi(x^{(i)}) \right) \quad \forall i \in \{1, \dots, m\}$$

Or in vector notation, let K be the $m \times m$ matrix with $K_{ij} = K(x^{(i)}, x^{(j)})$, we have

$$\beta := \beta + \alpha(\vec{y} - K\beta)$$

- Using this algorithm we can update the representation of β of vector θ with $O(m)$ time per update

- Now, we show that the knowledge of β is suffice to compute the prediction $\theta^T \phi(x)$

$$\theta^T \phi(x) = \sum_{i=1}^m \beta_i \phi(x^{(i)})^T \phi(x) = \sum_{i=1}^m \beta_i K(x^{(i)}, x)$$

- Fundamentally all we need to know about the feature map $\phi(\cdot)$ is encapsulated in th corresponding kernel function $K(\cdot, \cdot)$

- We have explicitly defined feature map ϕ , which induces the kernel function $K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$
- Once the kernel function is defined, the training algorithm can be written in terms of kernel function without referring to feature map $\phi(x)$
- We only need to ensure the existence of $\phi(x)$, but not necessary to write down ϕ
- What kind of functions $K(., .)$ can correspond to some feature map ϕ ? OR
- Can we tell if there is some feature map ϕ so that $K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle \forall x, z$?
- Soon we'll answer this question

- Suppose $x, z \in \mathbb{R}^d$, and let's consider a function $K(., .)$ defined as

$$K(x, z) = (x^T z)^2$$

- It can be written as

$$\begin{aligned} K(x, z) &= (x^T z)^2 \quad \because x^T z = \sum_{i=1}^d x_i z_i \\ &= \left(\sum_{i=1}^d x_i z_i \right) \left(\sum_{i=1}^d x_i z_i \right) \\ &= \sum_{i=1}^d \sum_{j=1}^d x_i x_j z_i z_j \\ &= \sum_{i,j=1}^d (x_i x_j) (z_i z_j) \\ &= \langle \phi(x), \phi(z) \rangle \end{aligned}$$

- The feature mapping for $d = 3$ is given by

$$\phi(x) = [x_1 x_1 \quad x_1 x_2 \quad x_1 x_3 \quad x_2 x_1 \quad x_2 x_2 \quad x_2 x_3 \quad x_3 x_1 \quad x_3 x_2 \quad x_3 x_3]^T$$

- Computing higher-dimensional $\phi(x)$ requires $O(d^2)$ time, whereas finding $K(x, z)$ takes only $O(d)$ time-linear in the dimension of input attribute
- Another kernel function, $K(., .)$ defined by

$$\begin{aligned} K(x, z) &= (x^T z + c)^2 \\ &= \sum_{i,j=1}^d (x_i x_j)(z_i z_j) + \sum_{i=1}^d (\sqrt{2cx_i})(\sqrt{2cz_i}) + c^2 \end{aligned}$$

- This is a kernel function that corresponds to the feature mapping (for $d = 3$)

$$\phi(x) = [x_1 x_1 \ x_1 x_2 \ x_1 x_3 \ x_2 x_1 \ x_2 x_2 \ x_2 x_3 \ x_3 x_1 \ x_3 x_2 \ x_3 x_3 \ \sqrt{2cx_1} \ \sqrt{2cx_2} \ \sqrt{2cx_3} \ c]^T$$

- In general, the kernel $K(x, z) = (x^T z + c)^k$ corresponds to a feature mapping to an $\binom{d+k}{k}$ feature space, corresponding of all monomials of the form $x_{i_1} x_{i_2} \dots x_{i_k}$ that are upto order k
- Despite working on $O(d^k)$ -dimensional space, computing $K(x, z)$ still takes only $O(d)$ time (and we never need to explicitly represent feature vector in high dimensional space)

Kernel as similarity metrics

- $K(x, z) = \begin{cases} \text{large} & \text{if } \phi(x) \text{ and } \phi(z) \text{ are close together} \\ \text{small} & \text{if } \phi(x) \text{ and } \phi(z) \text{ are far apart} \end{cases}$
- We can think of $K(x, z)$ as some measurement of how similar are $\phi(x)$ and $\phi(z)$, or of how similar are x and z
- Given this intuition about $K(x, z)$, lets assume that for some learning problem, we consider some function $K(x, z)$, which we may think might be a reasonable measure of how similar x and z are
- Lets assume,

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) = \begin{cases} \approx 1 & \text{when } x \text{ and } z \text{ are close} \\ \approx 0 & \text{when } x \text{ and } z \text{ are far apart} \end{cases}$$

- It is reasonable measure of x and z 's similarity
- This kernel is called the **Gaussian kernel**, and corresponds to infinite dimensional feature mapping ϕ

Necesssary conditions for valid kernels

- Given some function K , how can we tell if it is a valid kernel i.e., can we tell if there is some feature mapping ϕ so that $K(x, z) = \phi(x)^T \phi(z)$ for all x, z ?
- Suppose K is a valid kernel corresponding to some feature mapping ϕ
- Consider some finite set of points (not necessarily the training set) $\{x^{(1)}, \dots, x^{(m)}\}$
- Lets define a $m \times m$ square matrix K so that its (i, j) entry is given by $K_{ij} = K(x^{(i)}, x^{(j)})$, it is called the **Kernel matrix**

- Now, if K is a valid Kernel, then

$$K_{ij} = K(x^{(i)}, x^{(j)}) = \phi(x^{(i)})^T \phi(x^{(j)}) = K(x^{(j)}, x^{(i)}) = K_{ji}$$

hence K must be symmetric

- Let $\phi_k(x)$ denote the k -th coordinate of $\phi(x)$, then for any vector z , we have

$$\begin{aligned} z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\ &= \sum_i \sum_j z_i \phi(x^{(i)})^T \phi(x^{(j)}) z_j \\ &= \sum_i \sum_j z_i \sum_k \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\ &= \sum_k \sum_i \sum_j z_i \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\ &= \sum_k \left(\sum_i z_i \phi_k(x^{(i)}) \right)^2 \quad \because \sum_{i,j} a_i a_j = \left(\sum_i a_i \right)^2 \\ &\geq 0 \end{aligned}$$

- Since z is arbitrary, this shows that K is positive semi-definite

- So if K is a valid kernel, then the corresponding Kernel matrix $K \in \mathbb{R}^{m \times m}$ is symmetric positive semidefinite
- It is necessary and sufficient condition for K to be a valid kernel (also called Mercer kernel)

Theorem (Mercer)

Let $K : \mathbb{R}^d \times \mathbb{R}^d \mapsto \mathbb{R}$ be given. Then for K to be a valid (Mercer) kernel, it is necessary and sufficient that for any $\{x^{(1)}, \dots, x^{(m)}\}$, ($m < \infty$), the corresponding kernel matrix is symmetric positive semi-definite.

House Price Prediction: A Practical Example

Problem Setup

- **Problem:** Predict house price (y) based on living area (x) in sq. ft.
- **Training Data:** 3 houses with the following data:
 - House 1: $x^{(1)} = 1000$ sq ft, price = \$200,000
 - House 2: $x^{(2)} = 1500$ sq ft, price = \$350,000
 - House 3: $x^{(3)} = 2000$ sq ft, price = \$600,000
- We want to fit a cubic model: $y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$

Feature Map Construction

Mapping to Higher Dimensions

- Define feature map $\phi : \mathbb{R} \rightarrow \mathbb{R}^4$:

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

- For our training examples:

$$\phi(1000) = [1, 1000, 10^6, 10^9]^T$$

$$\phi(1500) = [1, 1500, 2.25 \times 10^6, 3.375 \times 10^9]^T$$

$$\phi(2000) = [1, 2000, 4 \times 10^6, 8 \times 10^9]^T$$

- Direct computation would require handling huge numbers!

The Kernel Trick

Avoiding Explicit Computation

- For cubic polynomial, the kernel function is:

$$K(x, z) = 1 + xz + (xz)^2 + (xz)^3$$

- **Solved Example:** Compute similarity between House 1 (1000) and House 2 (1500)

$$\langle x^{(1)}, x^{(2)} \rangle = 1000 \times 1500 = 1.5 \times 10^6$$

$$K(1000, 1500) = 1 + 1.5 \times 10^6 + (1.5 \times 10^6)^2 + (1.5 \times 10^6)^3$$

- Compare with direct computation:
 - **Direct:** Need 10^9 -dimensional vectors!
 - **Kernel:** Just 4 arithmetic operations!

Kernel Matrix Computation

- Compute kernel matrix K for all training examples:

$$K_{ij} = K(x^{(i)}, x^{(j)})$$

- For $i = 1, j = 2$ (1000 and 1500):

$$xz = 1.5 \times 10^6$$

$$\begin{aligned} K_{12} &= 1 + 1.5 \times 10^6 + (1.5 \times 10^6)^2 + (1.5 \times 10^6)^3 \\ &= 1 + 1.5 \times 10^6 + 2.25 \times 10^{12} + 3.375 \times 10^{18} \\ &\approx 3.375 \times 10^{18} \end{aligned}$$

- This single number captures the similarity between houses in feature space!

Dual Formulation Algorithm

Training with Kernel Trick

- Initialize $\beta = [0, 0, 0]^T$ (coefficient vector)
- Target prices: $\vec{y} = [200000, 350000, 600000]^T$
- Kernel matrix (simplified for illustration):

$$K \approx \begin{bmatrix} 1.001 \times 10^9 & 3.375 \times 10^{18} & 1.6 \times 10^{19} \\ 3.375 \times 10^{18} & 3.795 \times 10^{19} & 6.4 \times 10^{19} \\ 1.6 \times 10^{19} & 6.4 \times 10^{19} & 1.28 \times 10^{20} \end{bmatrix}$$

- Update β using: $\beta := \beta + \alpha(\vec{y} - K\beta)$ with $\alpha = 10^{-19}$

Sanjay Singh

Kernel Methods

First Iteration Calculation

- Initial $\beta = [0, 0, 0]^T$
- First update:

$$\beta := 0 + 10^{-19}(\vec{y} - K \cdot 0) = 10^{-19}\vec{y}$$

- $\beta_1 = 2 \times 10^{-14}, \beta_2 = 3.5 \times 10^{-14}, \beta_3 = 6 \times 10^{-14}$
- Now $\theta = \sum_{i=1}^3 \beta_i \phi(x^{(i)})$ without ever computing ϕ explicitly!
- For prediction at new house with $x = 1200$ sq ft:

$$\begin{aligned} \hat{y} &= \sum_{i=1}^3 \beta_i K(x^{(i)}, 1200) \\ &= \beta_1 K(1000, 1200) + \beta_2 K(1500, 1200) + \beta_3 K(2000, 1200) \end{aligned}$$

Sanjay Singh

Kernel Methods

Key Insights from Example

Why Kernels Are Powerful

- **Computational Savings:**
 - Direct approach: Store and compute with 10^9 dimensions
 - Kernel approach: Just compute xz (scalar) and powers
 - **Speedup:** $\frac{10^9}{4} = 250$ million times faster!
- **Memory Savings:**
 - Direct: Store 10^9 parameters θ
 - Kernel: Store only $m = 3$ coefficients β
- **Generalization:** This works for any dataset size - we only need m coefficients, not p parameters!

Sanjay Singh

Kernel Methods

Prediction for New House

Solved Example: 1800 sq ft House

- Predict price for $x = 1800$ sq ft
- Need $K(x^{(i)}, 1800)$ for each training point:

$$\begin{aligned} K(1000, 1800) &= 1 + 1.8 \times 10^6 + (1.8 \times 10^6)^2 + (1.8 \times 10^6)^3 \\ &\approx 5.832 \times 10^{18} \end{aligned}$$

$$\begin{aligned} K(1500, 1800) &= 1 + 2.7 \times 10^6 + (2.7 \times 10^6)^2 + (2.7 \times 10^6)^3 \\ &\approx 1.968 \times 10^{19} \end{aligned}$$

$$\begin{aligned} K(2000, 1800) &= 1 + 3.6 \times 10^6 + (3.6 \times 10^6)^2 + (3.6 \times 10^6)^3 \\ &\approx 4.666 \times 10^{19} \end{aligned}$$

- Final prediction:
 $\hat{y} = \beta_1(5.832 \times 10^{18}) + \beta_2(1.968 \times 10^{19}) + \beta_3(4.666 \times 10^{19})$
- **This captures cubic relationship without explicit high-dim computation!**

Sanjay Singh

Kernel Methods

Summary

What We Learned

- **Kernel Function:** $K(x, z) = \langle \phi(x), \phi(z) \rangle$ computes similarity in feature space
- **Key Insight:** We never need $\phi(x)$ explicitly - only dot products via kernel
- **Practical Benefit:**
 - From $O(p)$ computation to $O(d)$ computation
 - From storing p parameters to storing m coefficients
 - In our example: $p = 10^9, d = 1, m = 3 \rightarrow$ enormous savings!
- **Real-world Impact:** Kernels make high-dimensional learning feasible for:
 - Polynomial regression (like our example)
 - Support Vector Machines
 - Gaussian Processes
 - Many other ML algorithms

Kernel Trick: A General Principle

Application of Kernel Methods

- **Key Insight:** Kernels aren't limited to regression or SVMs
- **Fundamental Observation:**

If algorithm depends on inputs *only* via inner products $\langle x, z \rangle$

- **Then we can “kernelize” it:**

$$\langle x, z \rangle \rightarrow K(x, z) = \langle \phi(x), \phi(z) \rangle$$

- This substitution **magically** enables operation in high-dimensional feature space *without ever computing* $\phi(x)$ explicitly

Why “magically”?

We implicitly work in a transformed space while maintaining computational complexity proportional to the original space!

Beyond Regression: Kernel Perceptron

- **Example Application:** The Perceptron Algorithm
- Standard Perceptron update rule (error-driven):

$$\theta := \theta + y^{(i)}x^{(i)} \quad (\text{when mistake occurs})$$

- Parameter vector as linear combination of examples:

$$\theta = \sum_j \alpha_j y^{(j)} x^{(j)}$$

- Prediction function depends on inner products:

$$h(x) = \text{sign}(\langle \theta, x \rangle) = \text{sign} \left(\sum_j \alpha_j y^{(j)} \langle x^{(j)}, x \rangle \right)$$

- **Kernel Perceptron:** Replace $\langle x^{(j)}, x \rangle$ with $K(x^{(j)}, x)$:

$$h(x) = \text{sign} \left(\sum_j \alpha_j y^{(j)} K(x^{(j)}, x) \right)$$

Sanjay Singh

Kernel Methods

Power of the Kernel Trick

- **Universal Applicability:**
 - Linear regression
 - Support Vector Machines (SVMs)
 - **Perceptron**
 - Principal Component Analysis (PCA)
 - Fisher discriminant analysis
 - Many others...
- **Key Requirement:**
 - Algorithm must be expressible using only inner products $\langle x, z \rangle$

Coming Attractions

Many algorithms we'll study later are amenable to the kernel trick:

- Kernel PCA
- Kernel ridge regression
- Kernel Fisher discriminant

Take-home Message

The kernel trick is a general meta-technique: if you can write it with inner products, you can kernelize it!

Sanjay Singh

Kernel Methods